

**FUTURESTACK**

# Engineering Docs

Architecture, setup, data model, APIs and product flows — one reference for the whole codebase.

---

## CONTENTS

|                              |    |
|------------------------------|----|
| Project Overview             | 2  |
| 01 What's in the Product     | 3  |
| 02 Architecture              | 6  |
| 03 Local Setup               | 8  |
| 04 Environment Variables     | 10 |
| 05 Database Schema           | 12 |
| 06 Auth, Roles & Permissions | 16 |
| 07 API Reference             | 20 |
| 08 Backend Modules           | 29 |
| 09 Frontend Structure        | 37 |
| 10 Core Flows                | 39 |
| 11 Commerce & Payments       | 40 |
| 12 Admin CMS & Master Data   | 44 |
| 13 AI Content Generation     | 45 |
| 14 Deployment                | 46 |
| 15 Glossary                  | 48 |

# Project Overview

Owner: Ram · Last reviewed: TODO · Status: Completed

## What FutureStack is

FutureStack is a full-stack edtech platform. Students buy courses, watch video lessons in a fixed order, take quizzes, work on live projects, and receive a certificate when they finish. Behind the public site there is a multi-role operations panel where staff manage sales, content, trainers, support and refunds.

## Who uses it

| User type       | Role enum       | Where they log in           | What they do                                                          |
|-----------------|-----------------|-----------------------------|-----------------------------------------------------------------------|
| Student         | STUDENT         | / (embedded login) or OAuth | Browse, buy, learn, take tests, get certificates, post in discussions |
| Admin           | ADMIN           | /ops via /staff-login       | Full access — CMS, payments, master data, users                       |
| Trainer         | TRAINER         | /ops                        | Course content, student progress, doubts                              |
| Coordinator     | COORDINATOR     | /ops                        | Batch / student coordination                                          |
| Support         | SUPPORT         | /ops                        | Support tickets                                                       |
| Sales           | SALES           | /ops                        | Leads, sales targets, order follow-ups                                |
| Content Manager | CONTENT_MANAGER | /ops                        | Articles, pages, careers, blog                                        |

## Core capabilities

1. Course delivery — sections -> videos -> quizzes, with sequential unlocking and progress tracking. Certificate auto-issues on completion.
2. Quizzes / Skill Tests — one unified Quiz model serving both course-linked quizzes and standalone skill tests.
3. Commerce — cart, coupons, Razorpay checkout, multi-currency (INR / USD), refunds, invoices.
4. Admin CMS — legal pages (Terms, Privacy, Refund, Cookie), articles, career job postings, payment settings, master data.
5. Community & support — discussion forums, reviews, support tickets.
6. Live Projects — real project work attached to student profiles.
7. AI content — blog articles auto-generated via OpenAI.

## What FutureStack is NOT (scope boundaries)

- Not a live-class / video-conferencing product. Video is pre-recorded via VdoCipher.
- Not a marketplace — courses are first-party, there is no external instructor payout marketplace.
- TODO: add other explicit non-goals so scope creep is easy to reject.

## Primary user journeys

| Journey                          | Entry point               | Documented in                            |
|----------------------------------|---------------------------|------------------------------------------|
| Discover -> buy -> enroll        | /courses                  | 10 Core Flows / Purchase & Enrollment    |
| Learn -> complete -> certificate | /my-dashboard/my-courses  | 10 Core Flows / Learning & Certification |
| Take a skill test                | /my-dashboard/skill-tests | 10 Core Flows / Quiz & Skill Test        |
| Raise a support ticket           | /support                  | 10 Core Flows / Support Ticket           |
| Request a refund                 | /order-history            | 11 Commerce & Payments                   |

# 01 — What's in the Product

Features:

- Student site — course catalog, checkout, "my dashboard" (courses, live projects, skill tests, certificates)
- Ops consoles — role-based internal portals
- Payments — Razorpay checkout (domestic + international)
- Video — hosted lessons stream securely

Tech stack:

- Frontend: Next.js (React)
- Backend: NestJS API
- Database: PostgreSQL (Prisma ORM)

## Screenshots — live app

**FUTURE STACK** CODE. CREATE. CONQUER.

We are here to help you **LEARN, CREATE AND CONQUER**  
Build future-ready skills with industry-relevant technologies

Build Today for a **Bigger Tomorrow**

**Popular Courses**

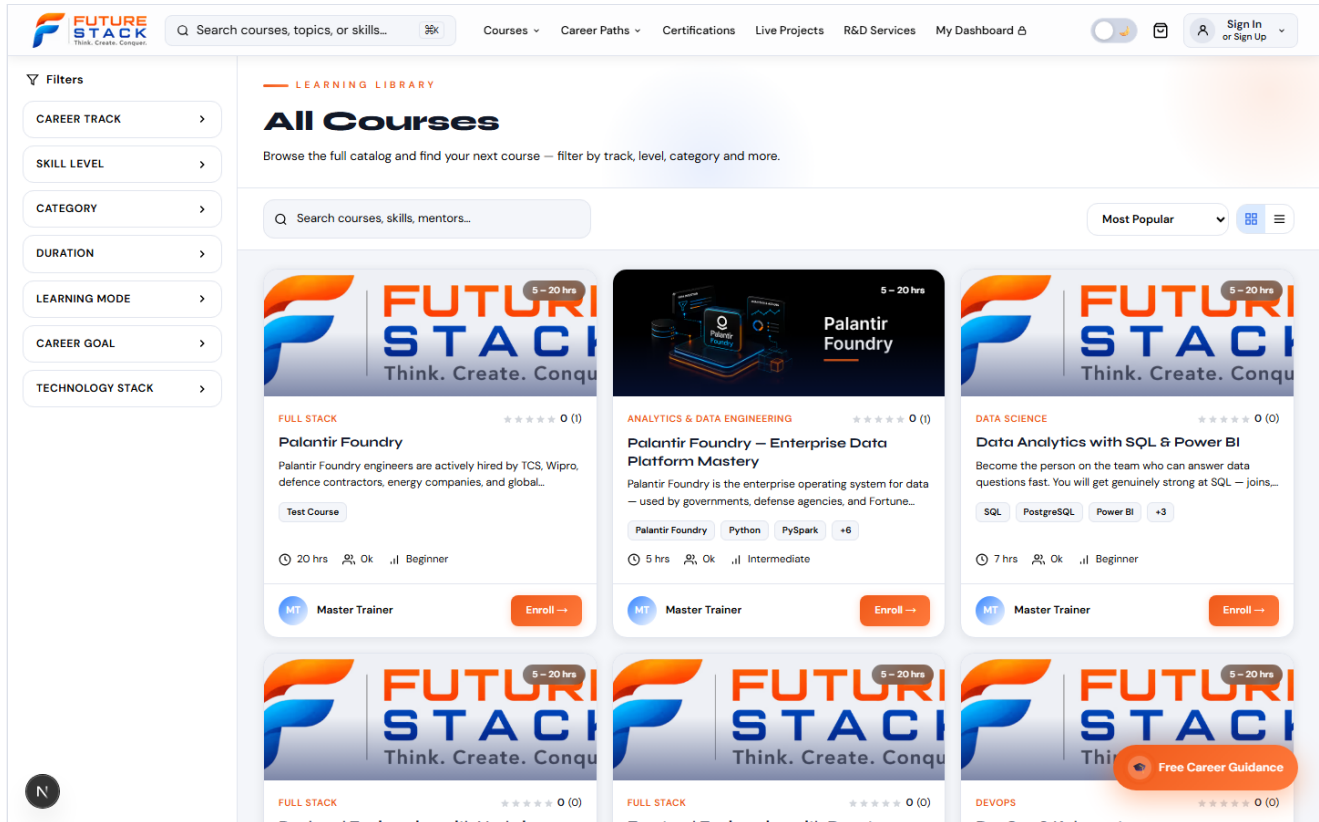
- Palantir Foundry — Enterpri...**  
Palantir Foundry Python  
PySpark  
High-demand enterprise skill — ...  
5 hrs Certificate
- MERN Stack Development**  
MongoDB Mongoose Express.js  
High-demand skill — average...  
17 hrs Certificate
- Data Analytics with SQL & P...**  
SQL PostgreSQL Power BI  
Data Analyst — the fastest path...  
7 hrs Certificate
- Frontend Engineering with R...**  
React TypeScript React Query  
Frontend Engineer — consistently...  
7 hrs Certificate
- DevOps & Kubernetes**  
Docker Kubernetes Helm  
Platform / DevOps Engineer — #18...  
7 hrs Certificate
- Backend Engineering with N...**  
Node.js TypeScript Express  
Backend Engineer — the core of...  
7 hrs Certificate
- Cloud Engineering with AWS**  
AWS VPC IAM  
Cloud Engineer — one of the mos...  
7 hrs Certificate
- Deep Learning & Neural Net...**  
Python PyTorch torchvision  
Deep learning specialists...  
8 hrs Certificate
- Machine Learning Engineeri...**  
Python scikit-learn XGBoost  
ML Engineer — average salary...  
8 hrs Certificate
- Python for Data Science**  
Python NumPy pandas  
The entry point to every data role...  
7 hrs Certificate

**Student Login**  
Access your dashboard & courses  
Email Address  
Enter your email  
Password  
Enter password  
Remember me Forgot Password?  
Login  
or continue with  
Continue with Gmail  
Don't have an account? Sign Up

**FREE COUNSELLING**  
**Not sure which course fits you?**  
Talk to a career expert for 15 minutes — get a personalised roadmap, fee & batch details. No cost, no obligation.  
Request a Free Callback  
Avg. response time: under 2 hours

**Quick Actions**  
Enroll Now Free Career Guidance

Home page — hero, popular courses, student login panel.



Course catalog (/courses) — filters, search, course cards.

## Local Setup

Prerequisites: Node.js, pnpm, Git.

```
# 1. Clone the repo
git clone https://github.com/futurestackdevfs/futurestack.git
cd futurestack

# 2. Install pnpm (if you don't have it)
npm install -g pnpm

# 3. Install all dependencies (monorepo - installs for both web + api)
pnpm install
```

Environment files — you'll need two .env files, get them from a team lead (or follow .env.example if one exists): apps/web/.env (frontend config), apps/api/.env (DB connection, JWT secret, Google OAuth, Razorpay, AWS/S3, VdoCipher, email keys, etc.).

Database setup (inside apps/api):

```
npx prisma generate      # generate the Prisma client
npx prisma migrate deploy # apply migrations
```

## Running the project

You'll need two terminals — frontend and backend run separately.

```
# Terminal 1 — Backend API (port 3002)
cd apps/api
pnpm run start:dev

# Terminal 2 — Frontend (port 3000)
cd apps/web
pnpm dev
```

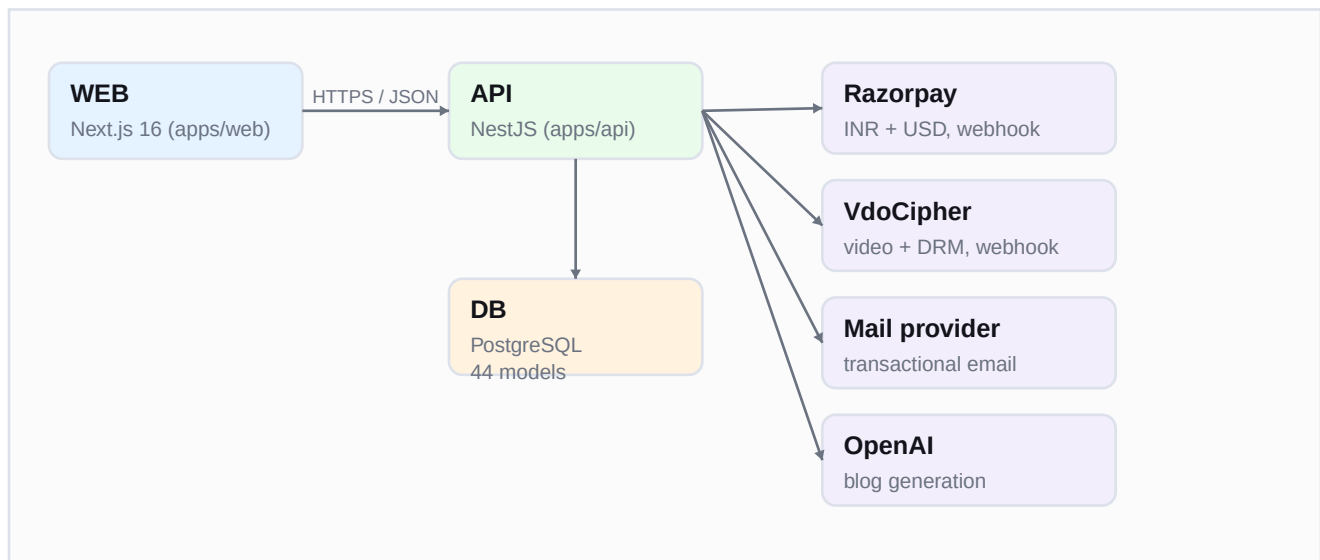
Open <http://localhost:3000> — the frontend automatically talks to the backend at <http://localhost:3002>.

```
pnpm test      # run backend tests
pnpm build     # production build
```

## 02 — Architecture

Owner: Ram · Last reviewed: TODO · Status: Completed

### System diagram



30 modules · 29 controllers · ~270 endpoints. Full request lifecycle below.

## Repo layout

```
FutureStack/
|-- apps/
|   |-- api/                # NestJS backend
|   |   |-- src/
|   |       |-- auth/  student/  courses/  sales/  admin/ ...
|   |       |-- common/      # guards, decorators, filters, pipes
|   |       |-- prisma/      # Prisma service + schema
|   |       |-- main.ts
|   |-- web/                # Next.js 16 frontend
|       |-- app/
|           |-- (student)/    # public site + my-dashboard
|           |-- ops/          # role-based staff consoles
|-- docs/                   # this documentation
```

## Backend module map

Modules are grouped by domain. Full detail per module is in 08 Backend Modules.

| Domain           | Modules                                                                    |
|------------------|----------------------------------------------------------------------------|
| Identity & users | auth, student                                                              |
| Learning         | courses, certificates, projects, discussion, reviews                       |
| Commerce         | cart, checkout, coupon, refund, invoices, payment-settings, admin-payments |
| Staff & ops      | admin, sales, sales-targets, coordinator, trainer, support                 |
| Content & CMS    | blog, careers, legal-pages, contact, ai                                    |
| Platform         | upload, vdocipher, mail, audit, common                                     |

## Request lifecycle

1. Next.js client calls the API with credentials (see 06 Auth, Roles & Permissions).
2. Global middleware / pipes run — validation, rate limiting.
3. JwtAuthGuard resolves the user; RolesGuard checks the role on the route.
4. Controller -> service -> Prisma -> PostgreSQL.
5. Side effects (mail, audit log, webhook dispatch) fire from the service layer.
6. Response returns to the client.

TODO: confirm whether a global response envelope and global exception filter exist. If not, log it in 15 Known Issues.

## Cross-cutting concerns

| Concern             | Where it lives              | Notes                             |
|---------------------|-----------------------------|-----------------------------------|
| Validation          | Global ValidationPipe       | DTOs use class-validator          |
| Auth guards         | common/ decorators + guards | @Auth(Role.X)                     |
| Rate limiting       | ThrottlerModule             | TODO: confirm single registration |
| Audit trail         | audit module                | Which actions are logged — TODO   |
| File uploads        | upload module               | Storage target — TODO             |
| Transactional email | mail module                 | Templates list — TODO             |

## External dependencies

| Service           | Used for                     | Failure impact         | Fallback                           |
|-------------------|------------------------------|------------------------|------------------------------------|
| Supabase Postgres | All persistence              | Total outage           | None                               |
| Razorpay          | Payments + webhooks          | No new purchases       | Manual enrollment — see 18 Runbook |
| VdoCipher         | Video playback + OTP         | Students cannot watch  | None                               |
| OpenAI            | Blog generation              | Blog automation pauses | Write manually                     |
| Mail provider     | OTP, receipts, notifications | Silent UX failures     | TODO                               |

# 03 — Local Setup

## Prerequisites

| Tool       | Version                 | Check          |
|------------|-------------------------|----------------|
| Node.js    | e.g. 20.x               | node -v        |
| npm / pnpm | TODO                    | npm -v         |
| PostgreSQL | 14+ (or Supabase cloud) | psql --version |
| Git        | any                     | git --version  |

## First-time setup

```
# 1. Clone
git clone <repo-url> FutureStack
cd FutureStack

# 2. Install (root – monorepo)
npm install

# 3. Environment files
cp apps/api/.env.example apps/api/.env
cp apps/web/.env.example apps/web/.env.local
# Fill both – see "04 Environment Variables"

# 4. Database
cd apps/api
npx prisma generate
npx prisma migrate dev
npx prisma db seed          # TODO: confirm seed script exists

# 5. Run
npm run start:dev          # API -> http://localhost:3001
cd ../web && npm run dev    # Web -> http://localhost:3000
```

## Seed data

| What gets seeded | Login | Notes                 |
|------------------|-------|-----------------------|
| Admin user       | —     | Needed to access /ops |
| Sample course    | —     |                       |
| Master data      | —     |                       |

## Useful commands

| Command                         | Directory  | What it does                 |
|---------------------------------|------------|------------------------------|
| npx prisma studio               | apps/api   | Browse the DB in a GUI       |
| npx prisma migrate dev --name x | apps/api   | Create + apply a migration   |
| npx prisma migrate reset        | apps/api   | Wipe and reseed — local only |
| npm run build                   | either app | Production build             |
| npm run test                    | apps/api   | Unit tests                   |

## Common setup errors

| Error | Cause | Fix |
|-------|-------|-----|
|-------|-------|-----|

|                               |                                     |                                                        |
|-------------------------------|-------------------------------------|--------------------------------------------------------|
| PrismaClient is unable to run | Client not generated                | <code>npx prisma generate</code>                       |
| EADDRINUSE :3001              | Port busy                           | Kill the process or change PORT                        |
| DB connection timeout         | Supabase URL / IP restrictions      | Check DATABASE_URL, check Supabase network settings    |
| 401 on every request          | Missing JWT_SECRET or cookie config | See 04 Environment Variables                           |
| CORS error in browser         | Frontend origin not allowed         | Check CORS config in <code>apps/api/src/main.ts</code> |

## Working against cloud vs local DB

Pointing DATABASE\_URL at the Supabase cloud database during local development adds network latency to every query. For day-to-day development, prefer a local Postgres instance.

# 04 — Environment Variables

Owner: Ram · Last reviewed: 2026-09-19 · Status: Restricted

No real secret values below — placeholders only.

## Confirmed used in code (`process.env.* grep`)

### apps/api/src

| Variable              | Notes                                                             |
|-----------------------|-------------------------------------------------------------------|
| DATABASE_URL          | Prisma connection string                                          |
| FRONTEND_URL          | Used in password-reset email links, CORS                          |
| NODE_ENV              | development / production                                          |
| PORT                  | Server port (3002 per <code>render.yaml</code> )                  |
| BLOG_AUTOGEN_CRON     | Cron schedule for AI blog generation                              |
| BLOG_INTERNAL_API_KEY | Backs BlogApiKeyGuard for <code>/internal/articles*</code> routes |

### apps/web

| Variable             | Notes                                                                                                                      |
|----------------------|----------------------------------------------------------------------------------------------------------------------------|
| API_URL              | Backend URL for the BFF proxy, server-only, never exposed to the browser. Defaults to <code>http://localhost:3002</code> . |
| NEXT_PUBLIC_BASE_URL | Public site URL                                                                                                            |
| NODE_ENV             | development / production                                                                                                   |

## Additional variables confirmed via render.yaml (not caught by a plain grep)

| Variable                                                            | Purpose                                                                                                                                                                                      |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DIRECT_URL                                                          | Prisma direct connection (bypasses connection pooler — needed for migrations against Supabase)                                                                                               |
| JWT_SECRET                                                          | Access token signing secret (Render: generateValue: true)                                                                                                                                    |
| JWT_EXPIRES_IN                                                      | Set to 7d in render.yaml — note this conflicts with a hardcoded value in auth.module.ts; the module's hardcoded value currently wins since it isn't reading this env var. Worth reconciling. |
| GOOGLE_CLIENT_ID /<br>GOOGLE_CLIENT_SECRET /<br>GOOGLE_CALLBACK_URL | Google OAuth                                                                                                                                                                                 |
| AGENTMAIL_API_KEY                                                   | Transactional email provider                                                                                                                                                                 |
| AGENTMAIL_INBOX_ID                                                  | Set to "FutureStack <onboarding@agentmail.to>" in render.yaml                                                                                                                                |
| CONTACT_EMAIL                                                       | Destination for contact-form submissions                                                                                                                                                     |
| VDOCIPHER_API_KEY                                                   | VdoCipher video hosting                                                                                                                                                                      |

Not yet confirmed but almost certainly required given the codebase (Razorpay integration is extensive): RAZORPAY\_KEY\_ID, RAZORPAY\_KEY\_SECRET, RAZORPAY\_WEBHOOK\_SECRET — not caught by the grep passes run so far (likely read via ConfigService inside RazorpayClientService, not audited directly). Confirm these three directly in RazorpayClientService before treating this list as complete.

## No .env.example exists — action item

Confirmed by repo-wide search: zero .env.example or .env.local.example files exist anywhere in the monorepo. This is tracked as 15 Known Issues FS-011. Until it's added, this page is the closest thing to a reference — keep it updated whenever a new ConfigService.get('X') or process.env.X is added.

## Deployment topology

Only apps/api has a deployment definition (render.yaml, single Render web service, futurestack-api, free plan). No service block exists for apps/web in this repo — it is deployed separately, consistent with code comments referencing "Vercel's LB" in the BFF proxy route. See 14 Deployment.

## 05 — Database Schema

Owner: Ram · Last reviewed: 2026-09-19 · Status:

44 models, 13 enums. Source: [apps/api/prisma/schema.prisma](#).

### Enums

| Enum           | Values                                                                                |
|----------------|---------------------------------------------------------------------------------------|
| ApprovalStatus | PENDING, APPROVED, REJECTED                                                           |
| CourseStatus   | DRAFT, ACTIVE, ARCHIVED                                                               |
| SkillLevel     | BEGINNER, INTERMEDIATE, ADVANCED                                                      |
| VideoStatus    | UPLOADING, PROCESSING, READY, FAILED                                                  |
| Role           | STUDENT, TRAINER, COORDINATOR, SUPPORT, ADMIN, CONTENT_MANAGER, SALES                 |
| Currency       | INR, USD                                                                              |
| OrderStatus    | CREATED, PAID, FAILED, CANCELLED, EXPIRED, REFUND_REQUESTED, REFUNDED                 |
| RefundStatus   | PENDING, PROCESSED, REJECTED                                                          |
| GatewayType    | DOMESTIC, INTERNATIONAL                                                               |
| DiscountType   | PERCENT, FLAT                                                                         |
| MessageTag     | DOUBT, TIP, ANNOUNCEMENT, RESOURCE                                                    |
| TicketStatus   | OPEN, PENDING, RESOLVED, CLOSED                                                       |
| TicketPriority | LOW, NORMAL, HIGH, URGENT                                                             |
| TicketCategory | PAYMENT, REFUND, COURSE_ACCESS, VIDEO, CERTIFICATE, ACCOUNT, BUG, LIVE_SESSION, OTHER |

### Models by domain

#### Identity

**User** — the single table for students AND all staff roles, split by role: Role. Notable fields: companyId (unique, staff-only login handle), passwordChangedAt (bumping this instantly invalidates every existing access + refresh token), mustChangePassword, trainerSharePercent (per-trainer override of the global revenue split), flaggedToCoordinator/flagReason. Relations fan out to nearly everything: enrollments, videoProgress, quizAttempts, coursesTaught/projectsTaught (as trainer), certificates, discussions, revenueLedger/payouts (as trainer), refreshTokens, reviewsGiven, cart/wishlist, orders, salesOrders/leads, salesTargets, invoices, refunds, supportTickets/assignedTickets.

**RefreshToken** — tokenHash (SHA-256 of the raw token, unique), userId, expiresAt, revokedAt. onDelete: Cascade from User.

## Learning content

Track -> TrackCourse (join, @@unique([trackId,courseId])) -> Course. Course — title unique, price/originalPrice/priceUsd/originalPriceUsd (all Float — see Red Flags below), status: CourseStatus, code unique (used in certificate IDs), whatYoullLearn/techStack (String[]), averageRating/reviewCount (denormalized). Section (title unique — global, not per-course) -> Video / Quiz. Video — vdoCipherId, durationSeconds, order, videoStatus. Quiz — sectionId: String? is the entire distinction between a course quiz and a standalone Skill Test: null means standalone (shown at /my-dashboard/skill-tests); non-null means embedded in a course. QuizQuestion — options: String[], correctIndex (never sent to the client before submission). QuizAttempt — score/totalQuestions/correctCount/isPassed/answers(Json); no unique constraint on (student, quiz) — unlimited retakes are possible by design. VideoProgress — uniqueSecsWatched (furthest position ever reached, not cumulative playtime), lastPositionSec, isCompleted. Enrollment — status: String @default("active") (NOT an enum — observed values "active", "refunded"), @@unique([studentId,courseId]), Cascade from both User and Course. Certificate — credentialId unique, format FS-<year>-<courseCode>-<seq>; @@unique([studentId,courseId]) is the real concurrency guard. RevenueLedger — one row per paid enrollment, gross/platformCut/trainerShare all Float, deleted on refund. Payout — status: String @default("PENDING") (not an enum).

## Discussion & reviews

CourseDiscussion / CourseDiscussionReply / DiscussionUpvote — tag: MessageTag, isPinned/isAnswered flags; upvotes unique per (user, message) and per (user, reply). Review — polymorphic: either courseId or projectId is set; one review per student per course/project.

## Commerce

Cart (1:1 with User) -> CartItem. Wishlist (1:1) -> WishlistItem. Coupon — code unique, discountType, value: Float, applicableCourseIds (empty = applies to everything), maxUses/usedCount, perUserLimit (default 1), validFrom/validUntil. CouponRedemption — @@unique([couponId,userId]) is what actually enforces perUserLimit. Order — currency, gatewayType, status, razorpayOrderId unique, gstPercent/gstAmount snapshotted at purchase time, full billing address fields, salespersonId (nullable). OrderItem — polymorphic course/project, priceAtPurchase: Float. PaymentSettings — singleton: domesticEnabled default true, internationalEnabled default false, trainerSharePercent default 50, gstPercent default 18, gstPercentUsd default 0, usdRate default 95. Invoice — invoiceNumber unique, 1:1 with Order. Refund — status: RefundStatus, razorpayRefundId?, initiatedById.

## Content & CMS

LegalPage — slug unique ("terms" | "refund-policy" | "cookie-policy" | "sitemap"). JobPosting — title, duration, description. BlogPost — id uses cuid() (every other model uses uuid() — inconsistent); status: String @default("draft"). HeroSlide — homepage carousel slides.

## Sales & leads

Lead — status: String @default("New") ("New"|"Interested"|"Converted"|"Dropped" — not an enum).  
 LeadFollowUp — action: String. SalesTarget — period: String ("monthly"|"quarterly"|"yearly"),  
 targetAmount/currentAmount (Float).

## Live Projects

Project — same shape as Course but for live projects. ProjectCurriculum -> ProjectCurriculumVideo — mirrors Section/Video but for projects, shares the VideoProgress model via projectCurriculumVideoId.

## Support

SupportTicket — number: Int @unique @default(autoincrement()) (human-readable SUP-0XXXXX ref), category, priority, status, context: Json?. SupportTicketMessage — isInternalNote: Boolean (staff-only notes not shown to the student).

## Platform

AuditLog — append-only, no FK relations (fully denormalized: actorEmail/actorName stored as text). changes: Json? holds {before?, after?, requested?}.

## Money fields — confirmed systemic issue

Every single money-bearing field in the schema is Float, not Decimal. No exceptions were found. This covers Course/Project pricing, Enrollment.amountPaid, RevenueLedger.gross/platformCut/trainerShare, Payout.amount, Coupon.value/minOrderAmount, Order.subtotal/discountAmount/gstPercent/gstAmount/totalAmount, OrderItem.priceAtPurchase, PaymentSettings rate fields, Lead.budget, SalesTarget amounts, Invoice fields, Refund.amount.

Mitigating factor found in code: coupon.service.ts computes discounts in integer minor units (toMinor = Math.round(major\*100)) specifically to avoid float error at the point of discount calculation. See 15 Known Issues FS-001.

## Status-like fields that are plain String, not enum

| Field                                             | Values (from comments/code)              | Risk                                                         |
|---------------------------------------------------|------------------------------------------|--------------------------------------------------------------|
| Enrollment.status                                 | "active", "refunded"                     | A typo'd value silently breaks course access checks          |
| Payout.status                                     | "PENDING" and presumably others          | Mirrors RefundStatus pattern but isn't wired to the enum     |
| Order.paymentMethod / settledCurrency / batchMode | free text                                | settledCurrency duplicates the Currency enum as a raw string |
| OrderItem.status                                  | "pending" "active" "refunded"            |                                                              |
| Lead.status                                       | "New" "Interested" "Converted" "Dropped" |                                                              |

|                         |                                                             |
|-------------------------|-------------------------------------------------------------|
| LeadFollowUp.action     | "scheduled" "rescheduled" "completed" "converted" "dropped" |
| BlogPost.status         | "draft" default, presumably "published"                     |
| CourseResource.fileType | "PDF" "ZIP" "FIGMA" "PPT" free text                         |
| SalesTarget.period      | "monthly" "quarterly" "yearly"                              |

See 15 Known Issues FS-002.

## onDelete: Cascade — confirmed cascade chains

| Relation                           | Rule           | Consequence                                           |
|------------------------------------|----------------|-------------------------------------------------------|
| Enrollment.student -> User         | Cascade        | Deleting a User deletes their course access history   |
| Certificate.student -> User        | Cascade        | Deleting a User deletes their earned certificates     |
| Order.user -> User                 | Cascade        | Deleting a User deletes their financial order history |
| Invoice.order/user -> Order/User   | Cascade (both) | Deleting a User or Order deletes GST invoices         |
| Refund.order/student -> Order/User | Cascade (both) | Deleting a User or Order deletes refund records       |
| OrderItem.order -> Order           | Cascade        | Expected/benign — line items belong to the order      |

Confirmed red flag: a User delete cascades all the way through to Order, Invoice, Certificate, and Refund. For a system issuing GST invoices and processing Razorpay refunds, this means an account-deletion path destroys financial/audit records that should be retained regardless of account status, and silently revokes proof of course completion. See 15 Known Issues FS-003.

Countering relation: Course.trainer -> User has no explicit onDelete (Prisma default = Restrict), so deleting a trainer with courses assigned is blocked at the DB level — inconsistent with the Cascade pattern used elsewhere.

## ERD generation

Not yet wired up. Add to apps/api/prisma/schema.prisma:

```
generator erd {
  provider = "prisma-erd-generator"
  output   = "../../../docs/assets/erd.svg"
}
```

## 06 — Auth, Roles & Permissions

Owner: Ram · Last reviewed: 2026-09-19 · Status: Completed

### Registration & login paths

Student registration (`AuthService.register()`) — `bcrypt.hash(password, 12)`, creates User with role: STUDENT, immediately returns an access token + sets refresh cookie. Throttled 3/60s.

Trainer self-registration (`registerTrainer()`) — creates role: TRAINER, `approvalStatus: PENDING`. Does not return an access token. Login is blocked until an admin approves:

```
if (user.role === Role.TRAINER && user.approvalStatus !== 'APPROVED') {  
  if (user.approvalStatus === 'REJECTED') throw new UnauthorizedException('...not  
approved');  
  throw new UnauthorizedException('...pending admin approval');  
}
```

Login (`validateUser()`) — looks up by email OR companyId (so staff can log in with either), `bcrypt.compare`, checks `isActive` and `passwordExpiresAt` (temp-password expiry). Returns null on any failure — the strategy converts every failure to a generic 401. Portal gate at the controller level:

```
if (dto.portal === 'student' && user.role !== Role.STUDENT) throw new  
UnauthorizedException('Please use the staff login page');  
if (dto.portal === 'ops' && user.role === Role.STUDENT) throw new  
UnauthorizedException('Please use the student login page');
```

Throttled 5/60s.

Google OAuth — student-only by design (comment: "staff roles are provisioned manually"). Requires `emailVerified` from the Google profile. Matches by `googleId`, else links an existing account by email, else creates a new STUDENT. The callback hands the access token to the frontend via the URL fragment (`#token=`), deliberately not a query string, to avoid it leaking into server logs/Referer headers.

Known leftover: `google.strategy.ts:29` has `console.log((profile as any)._json)` — logs the full raw Google profile to the server console. Should be removed. See 15 Known Issues.

Logout — revokes ALL refresh tokens for the user in one `deleteMany`, not just the current session's token. Clears the role-scoped cookie.

Password reset — `forgotPassword()` always returns the same generic message regardless of whether the email exists (anti-enumeration). Generates `crypto.randomBytes(32)`, stores the SHA-256 hash on the user row with a 1-hour expiry, emails a raw-token link. `resetPassword()` hashes the incoming token, matches, sets the new password, and stamps `passwordChangedAt` — this single field instantly invalidates every existing access token (checked via `iat` in `JwtStrategy`) and every refresh token (checked

via `createdAt < passwordChangedAt`), without deleting a single row.

## Token lifetimes — as configured today

| Token                | Configured lifetime | File                              | Note                                                                                                                      |
|----------------------|---------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Access token         | 20 minutes          | <code>auth.module.ts:15-29</code> | Code comment reads "TESTING ONLY — revert to 15m before committing/ deploying". Not reverted. See 15 Known Issues FS-004. |
| Refresh token        | 7 days              | <code>auth.service.ts:58</code>   | Matches the cookie <code>maxAge</code>                                                                                    |
| Password reset token | 1 hour              | <code>auth.service.ts:415</code>  |                                                                                                                           |

## Refresh token: storage, rotation, reuse detection

Stored as a SHA-256 hash only, never plaintext. Rotation uses a deliberate "soft rotation" design: the consumed token is marked `revokedAt` rather than deleted, and a revoke within the last 60 seconds is treated as a legitimate concurrent refresh (e.g. two tabs racing a 401) rather than replay. Only a token revoked more than 60 seconds ago is rejected as reuse.

This is a known, intentional trade-off with a real edge: an attacker replaying a stolen-but-already-used refresh token within 60 seconds of the legitimate rotation would succeed. Documented here rather than treated as a silent gap.

No scheduled cleanup job for expired `RefreshToken` rows — they're opportunistically deleted at the top of every new `createRefreshToken()` call (unlike `Order`, which does have a `@Cron`).

## Cookies — role-scoped, one name per role

```
fs_student_refresh, fs_admin_refresh, fs_trainer_refresh, fs_cm_refresh,  
fs_coordinator_refresh, fs_support_refresh, fs_sales_refresh
```

All `httpOnly`, secure in production, `sameSite: lax`, 7-day `maxAge`. On `/auth/refresh`, the role used for the new cookie comes from the DB-verified user, never the request's query param — explicit anti-spoofing: "prevents a client from lying about their role to get the wrong cookie."

## Frontend token storage — confirmed from code

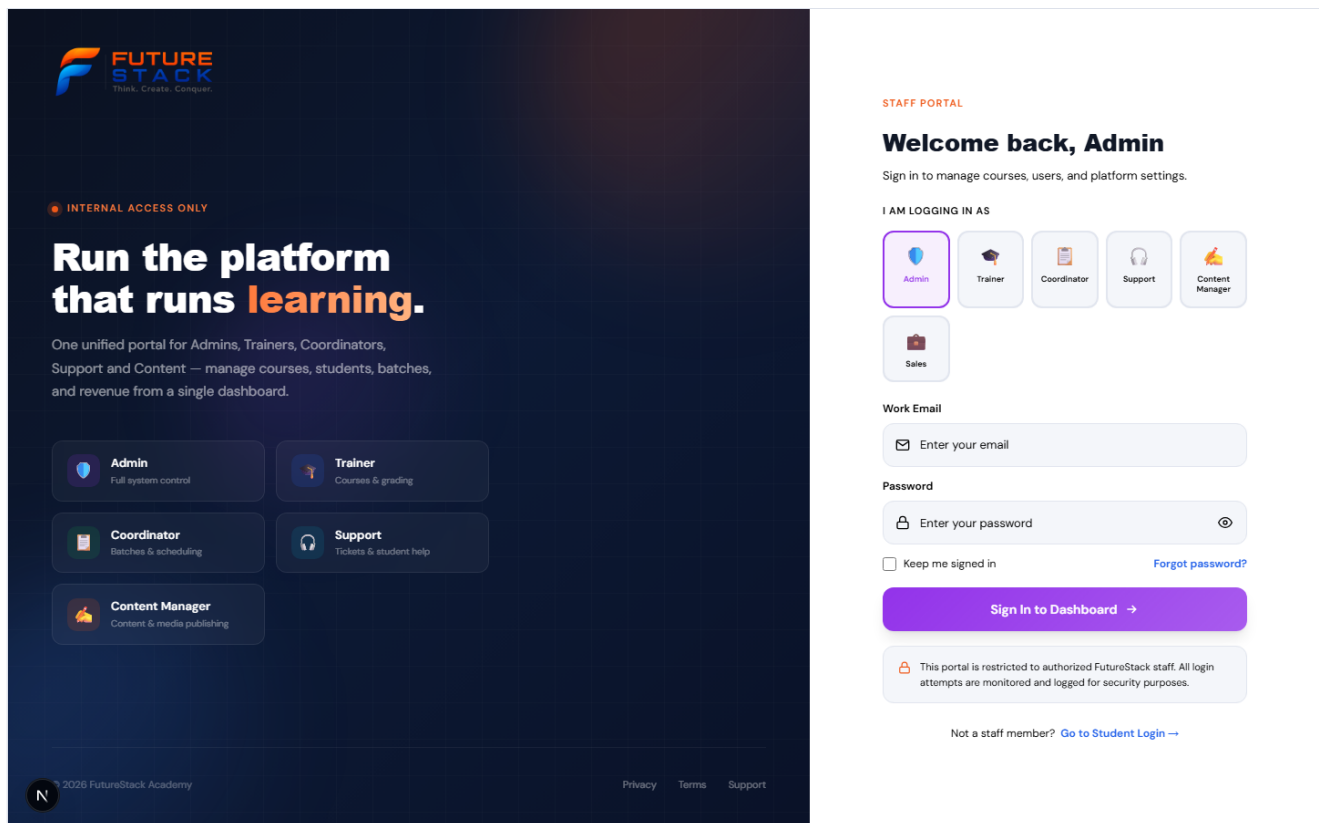
- Access token (JWT) lives in `localStorage`, under a key derived from `SHA-256(userId)` (`apps/web/app/auth/lib/token-store.ts`), with a fully separate `staff_` prefixed store so student and staff sessions never cross-contaminate in the same browser.
- The JWT is decoded client-side (`atob` on the payload) to populate React auth state — no server round trip needed for basic user info.

- The access token is also synced into an HttpOnly cookie (fs\_token / fs\_token\_staff) via POST /api/auth/set-token(-staff), specifically so the Next.js BFF proxy route can forward it as an Authorization header during SSR.
- On a fs:session-expired event, the client attempts a silent refresh before hard-logging-out, to avoid turning a recoverable 401 into an unwanted logout.

## Route protection — frontend vs backend

Frontend: apps/web/proxy.ts (Next.js middleware) does two things: (1) issues a per-request CSP header with a script nonce, (2) a coarse gate — if pathname.startsWith('/ops') and the fs\_token\_staff cookie is absent, redirect to /auth/staff-login. This is a UX redirect only, not real authorization — "the backend still enforces real authorization on every API call."

Backend (the actual enforcement): JwtAuthGuard + RolesGuard, composed via the @Auth(Role.X) decorator. JwtStrategy.validate() re-checks the user against the DB on every single request — rejects if !user.isActive, and rejects if the token's iat predates passwordChangedAt. RolesGuard must run after JwtAuthGuard, and allows any authenticated user through if the route has no @Roles() decorator — i.e. @Auth() with no arguments means "any logged-in user," not "public." Several controllers use this intentionally (discussion, support-student-side, upload/discussion).



Staff login (/auth/staff-login) — portal picker for Admin, Trainer, Coordinator, Support, Content Manager, Sales.

## Roles

| Role            | Console              | Notes                                                                                            |
|-----------------|----------------------|--------------------------------------------------------------------------------------------------|
| STUDENT         | /my-dashboard        | Default role on self-registration and OAuth                                                      |
| ADMIN           | /ops/admin           | Full access across nearly every controller                                                       |
| TRAINER         | /ops/trainer         | Requires admin approval (ApprovalStatus) before first login                                      |
| COORDINATOR     | /ops/coordinator     | Broad read access alongside its own console                                                      |
| SUPPORT         | /ops/support         | Tickets, student directory, read-only payments                                                   |
| SALES           | /ops/sales           | Leads, manual sales, own orders/targets                                                          |
| CONTENT_MANAGER | /ops/content-manager | Courses, projects, blog, careers, legal pages — same permissions as ADMIN on most content routes |

Staff accounts are created only via POST /admin/staff (@Auth(ADMIN)) — no self-registration path for staff roles other than TRAINER (which then requires approval).

## Public endpoints (deliberately unauthenticated)

Confirmed list: auth register/login/OAuth/refresh/forgot-reset-password; all courses.controller.ts public/\* routes; blog public article list/detail; legal-pages/:slug; payment-settings/public; contact (both forms); sales/public-leads; checkout/webhooks/razorpay (signature-protected instead); app.controller.ts root + /healthz; projects public list/detail/reviews; projects/:id/orders (guest project checkout entry point).

## Webhook authentication

| Webhook   | Route                                                                     | Verified how                                                                                                      |
|-----------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Razorpay  | POST /webhooks/razorpay                                                   | HMAC-SHA256 of raw body via RazorpayClient Service.validateWebhookSignature(), backed by RAZORPAY_WEBHOOK_SECRET. |
| VdoCipher | POST /admin/videos/webhook/* (7 routes) + /admin/videos/vdocipher-webhook | Guarded by VdoCipher WebhookGuard, not @Auth — correct pattern (shared secret, not user JWT).                     |

## Confirmed security posture (verified, not assumed)

| Control                       | Status                    |
|-------------------------------|---------------------------|
| Passwords hashed server-side  | bcrypt.hash(password, 12) |
| Refresh tokens hashed at rest | SHA-256, never plaintext  |

|                                             |                                                             |
|---------------------------------------------|-------------------------------------------------------------|
| Payment signature verification              | HMAC-SHA256, constant-time compare                          |
| Webhook raw-body signature check            | Razorpay confirmed; VdoCipher via shared-secret guard       |
| Global password-change session invalidation | via passwordChangedAt                                       |
| Anti-enumeration on password reset          | generic response always                                     |
| OAuth token leak via Referer                | mitigated — token passed via URL fragment, not query string |
| Access token lifetime matches intent        | 20m (see FS-004)                                            |
| Refresh reuse detection                     | intentional 60-second blind window by design                |
| Debug logging of OAuth profile              | console.log in google.strategy.ts:29                        |

## 07 — API Reference

Owner: Ram · Last reviewed: 2026-09-19 · Status: Completed

Confirmed via: `grep -rHE "@(Get|Post|Patch|Put|Delete)\(" apps/api/src --include="*.controller.ts" | wc -l`  
`=> 270.`

Swagger is still worth setting up so this table generates itself going forward — but everything below is real, current, and verified against the code.

### admin/admin.controller.ts — @Controller('admin')

| Method | Route                       | Notes                                                    |
|--------|-----------------------------|----------------------------------------------------------|
| GET    | /admin/integrations         | @Auth(ADMIN)                                             |
| GET    | /admin/audit-logs           | @Auth(ADMIN)                                             |
| GET    | /admin/stats                | @Auth(ADMIN,CONTENT_MANAGER,COORDINATOR)                 |
| GET    | /admin/trainers             | @Auth(ADMIN,CONTENT_MANAGER,COORDINATOR)                 |
| GET    | /admin/trainers/pending     | @Auth(ADMIN,CONTENT_MANAGER,COORDINATOR)                 |
| GET    | /admin/trainers/approved    | @Auth(ADMIN,CONTENT_MANAGER,COORDINATOR)                 |
| POST   | /admin/trainers/:id/approve | @Auth(ADMIN,CONTENT_MANAGER),<br>@Audit(APPROVE,Trainer) |
| POST   | /admin/trainers/:id/reject  | @Auth(ADMIN,CONTENT_MANAGER),<br>@Audit(REJECT,Trainer)  |
| POST   | /admin/staff                | @Throttle(10/60s), @Auth(ADMIN),<br>@Audit(CREATE,Staff) |
| GET    | /admin/users                | @Auth(ADMIN,COORDINATOR)                                 |

|        |                                                           |                                                               |
|--------|-----------------------------------------------------------|---------------------------------------------------------------|
| GET    | /admin/search                                             | @Auth(ADMIN)                                                  |
| POST   | /admin/users/:id/regenerate-password                      | @Throttle(10/60s), @Auth(ADMIN),<br>@Audit(UPDATE,User)       |
| GET    | /admin/sales-dashboard                                    | @Auth(ADMIN)                                                  |
| PATCH  | /admin/trainers/:id/share                                 | @Throttle(10/60s), @Auth(ADMIN),<br>@Audit(UPDATE,Trainer)    |
| POST   | /admin/videos/upload-credentials                          | @Throttle(10/60s),<br>@Auth(ADMIN,CONTENT_MANAGER)            |
| GET    | /admin/videos/:videoid/status                             | @Auth(ADMIN,CONTENT_MANAGER)                                  |
| POST   | /admin/videos/vdocipher-webhook                           | VdoCipherWebhookGuard,<br>@Throttle(30/60s)                   |
| POST   | /admin/enrollments                                        | @Throttle(10/60s), @Auth(ADMIN),<br>@Audit(CREATE,Enrollment) |
| GET    | /admin/enrollments                                        | @Auth(ADMIN,COORDINATOR)                                      |
| DELETE | /admin/enrollments/:id                                    | @Auth(ADMIN),<br>@Audit(DELETE,Enrollment)                    |
| GET    | /admin/reports/revenue.csv, leads.csv,<br>conversions.csv | @Auth(ADMIN,COORDINATOR)                                      |
| POST   | /admin/videos/webhook/* (6 routes)                        | VdoCipherWebhookGuard                                         |

### admin-payments/admin-payments.controller.ts — @Controller('admin/payments')

| Method | Route                    | Notes                                                    |
|--------|--------------------------|----------------------------------------------------------|
| GET    | /admin/payments          | @Auth(ADMIN,COORDINATOR) —<br>query: status,page,perPage |
| GET    | /admin/payments/trainers | @Auth(ADMIN,COORDINATOR) —<br>trainer revenue breakdown  |

### app.controller.ts — @Controller()

| Method | Route    | Notes                                                              |
|--------|----------|--------------------------------------------------------------------|
| GET    | /        | none — root hello                                                  |
| GET    | /healthz | @SkipThrottle() — health check for<br>Render + admin System Health |

### auth/auth.controller.ts — @Controller('auth')

| Method | Route                  | Notes                                                           |
|--------|------------------------|-----------------------------------------------------------------|
| POST   | /auth/register         | @Throttle(3/60s) — student registration,<br>sets refresh cookie |
| POST   | /auth/register-trainer | @Throttle(3/60s) — pending approval                             |
| POST   | /auth/login            | @Throttle(5/60s), LocalAuthGuard —<br>portal-gated              |

|      |                                     |                                           |
|------|-------------------------------------|-------------------------------------------|
| GET  | /auth/me                            | JwtAuthGuard                              |
| GET  | /auth/google, /auth/google/callback | GoogleAuthGuard                           |
| POST | /auth/refresh                       | none (cookie-based) — query: role         |
| POST | /auth/logout                        | JwtAuthGuard — revokes ALL refresh tokens |
| POST | /auth/forgot-password               | @Throttle(3/900s)                         |
| POST | /auth/reset-password                | @Throttle(5/900s)                         |
| POST | /auth/set-password                  | @Throttle(5/900s), JwtAuthGuard           |

### blog/blog.controller.ts — @Controller()

| Method                    | Route                                           | Notes                                   |
|---------------------------|-------------------------------------------------|-----------------------------------------|
| POST                      | /internal/articles, /internal/articles/generate | BlogApiKeyGuard                         |
| POST                      | /articles/admin/generate                        | @Auth(ADMIN,CONTENT_MANAGER)            |
| PATCH                     | /internal/articles/:id/publish                  | BlogApiKeyGuard                         |
| PATCH                     | /articles/admin/:id/publish                     | @Auth(ADMIN,CONTENT_MANAGER)            |
| POST/<br>PATCH/<br>DELETE | /articles/admin, /articles/admin/:id            | @Auth(ADMIN,CONTENT_MANAGER),<br>@Audit |
| GET                       | /articles, /articles/:slug                      | public, cache 60s/300s                  |
| GET                       | /articles/admin                                 | @Auth(ADMIN,CONTENT_MANAGER)            |

BlogApiKeyGuard is registered in app.module.ts:93 — confirms the /internal/\* routes ARE protected, just not via @Auth/@Public.

### careers/careers.controller.ts — @Controller()

| Method          | Route                     | Notes                                   |
|-----------------|---------------------------|-----------------------------------------|
| GET             | /careers/jobs             | public, cached                          |
| GET             | /admin/careers/jobs       | @Auth(ADMIN,CONTENT_MANAGER)            |
| POST/<br>DELETE | /admin/careers/jobs(/:id) | @Auth(ADMIN,CONTENT_MANAGER),<br>@Audit |

### cart/cart.controller.ts — @Controller('cart')

| Method                  | Route                          | Notes                                                                                 |
|-------------------------|--------------------------------|---------------------------------------------------------------------------------------|
| GET/<br>POST/<br>DELETE | /cart, /cart/items(/:courseId) | @Auth(STUDENT)                                                                        |
| POST                    | /cart/merge                    | @Auth(STUDENT) — implemented but no frontend code calls it; see 09 Frontend Structure |

POST/DELETE /cart/apply-coupon, /cart/coupon @Auth(STUDENT)

### cart/wishlist.controller.ts — @Controller('wishlist')

| Method          | Route                                | Notes          |
|-----------------|--------------------------------------|----------------|
| GET/POST/DELETE | /wishlist, /wishlist/items/:courseId | @Auth(STUDENT) |
| POST            | /wishlist/:courseId/move-to-cart     | @Auth(STUDENT) |

### certificates/certificates.controller.ts — @Controller('certificates')

| Method | Route                                        | Notes                  |
|--------|----------------------------------------------|------------------------|
| GET    | /certificates/my, /certificates/my/:courseId | @Auth(STUDENT)         |
| GET    | /certificates/recent                         | public, cache 60s/300s |
| POST   | /certificates/claim/:courseId                | @Auth(STUDENT)         |

### checkout/checkout.controller.ts + webhook.controller.ts

| Method | Route                  | Notes                                                          |
|--------|------------------------|----------------------------------------------------------------|
| POST   | /checkout/create-order | @Throttle(5/60s), @Auth(STUDENT)                               |
| POST   | /checkout/verify       | @Auth(STUDENT)                                                 |
| POST   | /checkout/cancel       | @Throttle(10/60s), @Auth(STUDENT)                              |
| POST   | /webhooks/razorpay     | none — signature-verified against raw body; always returns 200 |

### contact/contact.controller.ts — @Controller('contact')

| Method | Route                | Notes                                               |
|--------|----------------------|-----------------------------------------------------|
| POST   | /contact/rnd-inquiry | @Throttle(5/60s), 202 — public R&D inquiry form     |
| POST   | /contact/message     | @Throttle(5/60s), 202 — public general contact form |

### coordinator/coordinator.controller.ts — @Controller('coordinator'), class @Auth(COORDINATOR, ADMIN)

| Method | Route                                                                           | Notes |
|--------|---------------------------------------------------------------------------------|-------|
| GET    | /coordinator/dashboard, /batches, /students, /escalations, /trainers, /payments |       |
| PATCH  | /coordinator/students/:id/flag, /escalations/:id                                |       |

**coupon/coupon.controller.ts — @Controller('admin/coupons')**

| Method                            | Route                         | Notes                                      |
|-----------------------------------|-------------------------------|--------------------------------------------|
| POST/<br>GET/<br>PATCH/<br>DELETE | /admin/coupons/:id)           | @Auth(ADMIN), @Audit on writes             |
| GET                               | /admin/coupons/:id/sales      | @Auth(ADMIN)                               |
| PATCH                             | /admin/coupons/:id/deactivate | @Auth(ADMIN),<br>@Audit(DEACTIVATE,Coupon) |

**courses/courses.controller.ts — @Controller('courses') — 43 endpoints**

| Method                            | Route                                                                                                                                       | Notes                                             |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| POST/<br>GET/<br>PATCH/<br>DELETE | tracks, hero-slides, sections, videos, quizzes, quiz questions, resources                                                                   | @Auth(ADMIN,CONTENT_MANAGER),<br>@Audit on writes |
| GET                               | public/featured-courses, public/featured-tracks, public/tracks/:id/courses, public/cards, public/slug/:slug, public/related/:id, public/:id | public, cached                                    |
| GET                               | public/videos/:videoId/verification                                                                                                         | @Throttle(10/60s) — preview video OTP             |
| GET                               | /courses/search                                                                                                                             | @Throttle(30/60s), cache 30s/300s                 |
| POST/<br>GET                      | /courses                                                                                                                                    | @Auth(ADMIN,CONTENT_MANAGER),<br>@Audit on create |
| GET                               | /courses/trainer/revenue, /trainer/payouts                                                                                                  | @Auth(TRAINER)                                    |
| GET/<br>PATCH/<br>DELETE          | /courses/:id                                                                                                                                | @Auth(ADMIN,CONTENT_MANAGER)                      |
| GET                               | /courses/:courseId/overview                                                                                                                 | @Throttle(60/60s), cached                         |
| POST/<br>DELETE                   | /courses/:id/tracks/:trackId                                                                                                                | @Auth(ADMIN,CONTENT_MANAGER)                      |
| PUT                               | /courses/:id/career-path                                                                                                                    | @Auth(ADMIN,CONTENT_MANAGER)                      |
| POST                              | /courses/:courseId/sections, /resources                                                                                                     | @Auth(ADMIN,CONTENT_MANAGER)                      |

File has an explicit ordering comment: static-prefix routes (tracks, sections, videos, quizzes, resources, public, trainer, reorder-featured) must be declared before bare :id routes, or Express treats the prefix as the id param.

**discussion/discussion.controller.ts — @Controller('discussion')**

| Method       | Route                 | Notes                     |
|--------------|-----------------------|---------------------------|
| GET/<br>POST | /discussion/:courseId | @Auth() any authenticated |

|                  |                                           |                                   |
|------------------|-------------------------------------------|-----------------------------------|
| PATCH/<br>DELETE | /discussion/:courseId/messages/:messageId | @Auth()                           |
| PATCH            | .../pin, .../answer                       | @Auth() — role-checked in service |
| POST/<br>DELETE  | .../replies, .../replies/:replyId         | @Auth()                           |
| POST             | .../upvote (message + reply)              | @Auth()                           |

@Auth() with no role list means "any authenticated user," not "public" — pin/answer/delete permissions are enforced inside the service by comparing role/ownership.

### **invoices/invoices.controller.ts — @Controller('invoices')**

| Method | Route                       | Notes                                                        |
|--------|-----------------------------|--------------------------------------------------------------|
| GET    | /invoices                   | @Auth(ADMIN,COORDINATOR,SALES)                               |
| GET    | /invoices/order/:orderId    | @Auth(ADMIN,COORDINATOR,SALES,STUDENT) — student can see own |
| POST   | /invoices/generate/:orderId | @Auth(ADMIN,COORDINATOR,SALES)                               |

### **legal-pages/legal-pages.controller.ts — @Controller()**

| Method        | Route                      | Notes                         |
|---------------|----------------------------|-------------------------------|
| GET/<br>PATCH | /admin/legal-pages(/:slug) | @Auth(ADMIN), @Audit on write |
| GET           | /legal-pages/:slug         | public, cached                |

### **payment-settings/payment-settings.controller.ts — @Controller()**

| Method        | Route                    | Notes                                     |
|---------------|--------------------------|-------------------------------------------|
| GET/<br>PATCH | /admin/payment-settings  | @Auth(ADMIN), @Audit on write — singleton |
| GET           | /payment-settings/public | public, cached 120s/600s                  |

### **projects/projects.controller.ts — @Controller('projects') — 25 endpoints**

| Method           | Route                                       | Notes                                                   |
|------------------|---------------------------------------------|---------------------------------------------------------|
| GET              | /projects, /projects/:id                    | public, cached                                          |
| GET              | /projects/admin/all, /admin/orders          | @Auth(ADMIN,CONTENT_MANAGER)                            |
| PATCH            | /projects/admin/orders/:orderId/status      | @Auth(ADMIN,CONTENT_MANAGER)                            |
| POST/<br>DELETE  | curriculum sections/videos                  | @Auth(ADMIN,CONTENT_MANAGER)                            |
| POST/<br>PUT/GET | /projects/:id/reviews(/:reviewId)/my-review | @Auth(STUDENT) for writes, public for GET               |
| POST             | /projects/:id/orders                        | none — deliberately public (guest checkout entry point) |

|                           |                                                             |                              |
|---------------------------|-------------------------------------------------------------|------------------------------|
| POST/<br>PATCH/<br>DELETE | /projects, /projects/:id, /curriculum                       | @Auth(ADMIN,CONTENT_MANAGER) |
| POST                      | demo-video / curriculum video upload-credentials + complete | @Auth(ADMIN,CONTENT_MANAGER) |
| GET                       | /projects/:id/orders                                        | @Auth(ADMIN,CONTENT_MANAGER) |

### refund/refund.controller.ts — @Controller('admin/refunds')

| Method | Route                     | Notes                                  |
|--------|---------------------------|----------------------------------------|
| POST   | /admin/refunds            | @Auth(ADMIN),<br>@Audit(REFUND,Refund) |
| GET    | /admin/refunds/:id        | @Auth(ADMIN)                           |
| PATCH  | /admin/refunds/:id/reject | @Auth(ADMIN),<br>@Audit(REJECT,Refund) |

### reviews/reviews.controller.ts — @Controller()

| Method       | Route                                  | Notes                   |
|--------------|----------------------------------------|-------------------------|
| POST/<br>PUT | /courses/:courseId/reviews/:reviewId   | @Auth(STUDENT)          |
| GET          | /courses/:courseId/reviews, /my-review | public / @Auth(STUDENT) |
| GET          | /trainer/project-reviews               | @Auth(TRAINER)          |

### sales-targets/sales-targets.controller.ts — @Controller('sales-targets'), class @Auth(SALES, ADMIN, COORDINATOR)

| Method                    | Route                  | Notes                                        |
|---------------------------|------------------------|----------------------------------------------|
| GET                       | /sales-targets         | own for SALES, all for ADMIN/<br>COORDINATOR |
| POST/<br>PATCH/<br>DELETE | /sales-targets/:id     | @Auth(ADMIN,COORDINATOR)                     |
| POST                      | /sales-targets/refresh | class-level — recompute currentAmount        |

### sales/public-leads.controller.ts — @Controller('public-leads')

| Method | Route         | Notes                                                                             |
|--------|---------------|-----------------------------------------------------------------------------------|
| POST   | /public-leads | @Throttle(10/60s), no auth — public<br>career-guidance form -> unassigned<br>Lead |

### sales/sales.controller.ts — @Controller('sales'), class @Auth(SALES, ADMIN, COORDINATOR)

| Method                    | Route                                                                        | Notes                       |
|---------------------------|------------------------------------------------------------------------------|-----------------------------|
| GET                       | /sales/dashboard, /courses, /students, /orders(/pending)(/:id), /leads(/:id) | class-level                 |
| POST                      | /sales/sale                                                                  | @Throttle(20/60s)           |
| POST                      | /sales/orders/:id/confirm-payment                                            | @Throttle(30/60s)           |
| POST/<br>PATCH/<br>DELETE | /sales/leads(/:id)                                                           | @Throttle(30/60s) on create |

### student/student.controller.ts — @Controller('student')

| Method | Route                                                       | Notes                                                |
|--------|-------------------------------------------------------------|------------------------------------------------------|
| GET    | /student/dashboard, /courses/:courseId, /profile, /orders   | @Auth(STUDENT)                                       |
| POST   | /student/videos/:videoId/progress                           | @Throttle(120/60s), @Auth(STUDENT)                   |
| GET    | quizzes/standalone, quizzes/mine, quizzes/:quizId/questions | @Auth(STUDENT)                                       |
| POST   | /student/quizzes/:quizId/submit                             | @Throttle(10/60s), @Auth(STUDENT)                    |
| GET    | /student/videos/:videoId/otp, project-videos/:videoId/otp   | @Auth(STUDENT)                                       |
| POST   | /student/project-videos/:videoId/progress                   | @Throttle(120/60s), @Auth(STUDENT)                   |
| GET    | /student/projects/:projectId/progress, /my-projects         | @Auth(STUDENT)                                       |
| PUT    | /student/profile                                            | @Auth(STUDENT)                                       |
| POST   | /student/avatar                                             | @Throttle(5/60s), @Auth(STUDENT) — image only, 512KB |

### support/support.controller.ts — @Controller('support') — 23 endpoints

| Method         | Route                                                         | Notes                                             |
|----------------|---------------------------------------------------------------|---------------------------------------------------|
| POST/<br>GET   | /support/tickets(/:id)(/messages)(/close)                     | @Auth() — ownership checked in service            |
| GET            | /support/staff/tickets, /stats, /agents, /tickets/:id         | @Auth(SUPPORT,ADMIN)                              |
| POST/<br>PATCH | /support/staff/tickets/:id(/messages)                         | @Auth(SUPPORT,ADMIN)                              |
| GET            | /support/staff/students, /all-students, /students/:id/profile | @Auth(SUPPORT,ADMIN)                              |
| POST           | /support/staff/students/:id/send-password-reset               | @Auth(SUPPORT,ADMIN)                              |
| GET            | /support/staff/ratings(/:type/:id), /payments(/stats)(/:id)   | @Auth(SUPPORT,ADMIN)                              |
| GET/<br>POST   | /support/staff/inbox/threads(/:id)(/reply)                    | @Auth(SUPPORT,ADMIN) — every email, ticket or not |

**trainer/trainer.controller.ts — @Controller('trainer'), class @Auth(TRAINER) — 9 endpoints**

| Method | Route                                                                           | Notes             |
|--------|---------------------------------------------------------------------------------|-------------------|
| GET    | /trainer/dashboard, /revenue, /students, /doubts, /reviews, /projects, /profile |                   |
| POST   | /trainer/flag-student                                                           |                   |
| PUT    | /trainer/profile                                                                |                   |
| POST   | /trainer/avatar                                                                 | image-only, 512KB |

**upload/upload.controller.ts — @Controller('upload')**

| Method | Route              | Notes                                                       |
|--------|--------------------|-------------------------------------------------------------|
| POST   | /upload/discussion | @Auth() any authenticated — 5MB                             |
| POST   | /upload/image      | @Auth(ADMIN,CONTENT_MANAGER)<br>— query: folder; 5MB        |
| POST   | /upload/resource   | @Auth(ADMIN,CONTENT_MANAGER)<br>— wide type allowlist, 50MB |

No separate vdocipher controller exists. Video OTP issuance lives in student.controller.ts (GET /student/videos/:videoId/otp, GET /student/project-videos/:videoId/otp) and public preview OTP in courses.controller.ts (GET /courses/public/videos/:videoId/verification); inbound VdoCipher webhooks live in admin.controller.ts.

**Generate this automatically going forward**

```
cd apps/api && npm i @nestjs/swagger
```

```
// apps/api/src/main.ts
const config = new DocumentBuilder()
  .setTitle('FutureStack API')
  .setVersion('1.0')
  .addBearerAuth()
  .addCookieAuth('fs_student_refresh')
  .build();
SwaggerModule.setup('api/docs', app, SwaggerModule.createDocument(app, config));
```

Then annotate controllers with @ApiTags/@ApiOperation/@ApiResponse as you touch them — a PR that adds an endpoint without one doesn't merge.

## 08 — Backend Modules

Owner: Ram · Last reviewed: TODO · Status: Draft

This section is split into six sub-pages by domain, matching the grouping in 02 Architecture: Identity & Users, Learning, Commerce, Staff & Operations, Content & CMS, Platform.

### Identity & Users

Modules: auth, student

#### auth

Purpose: Authenticate students and staff, issue and rotate tokens, handle OAuth and password reset.

Owns models: User (shared), refresh token / session model — TODO

Depends on: mail (OTP, reset links), common (guards, decorators)

| Method | Route                                  | Auth   | Purpose                                       |
|--------|----------------------------------------|--------|-----------------------------------------------|
| POST   | /auth/register                         | Public | Student signup                                |
| POST   | /auth/login                            | Public | Student login                                 |
| POST   | /auth/staff-login                      | Public | Staff login — TODO:<br>confirm separate route |
| POST   | /auth/refresh                          | Cookie | Rotate refresh, issue<br>access               |
| POST   | /auth/logout                           | Auth   | Invalidate refresh token                      |
| GET    | /auth/google(/callback)                | Public | OAuth start / callback                        |
| POST   | /auth/forgot-password, /reset-password | Public | Reset flow                                    |

- Staff accounts are never self-registered — created by ADMIN only. TODO: confirm.
- A student logging in via OAuth with an existing email must link, not duplicate, the account. TODO: confirm behaviour.
- Failed login attempts should be rate limited — see 06 Auth, Roles & Permissions.
- TODO: email verification required before purchase?

Files: apps/api/src/auth/auth.controller.ts, auth.service.ts, strategies/, dto/

Known issues: TODO

#### student

Purpose: Student-facing profile and dashboard aggregation.

Owns models: TODO

Depends on: courses, certificates, projects

| Method | Route              | Auth    | Purpose              |
|--------|--------------------|---------|----------------------|
| GET    | /student/me        | STUDENT | Profile              |
| PATCH  | /student/me        | STUDENT | Update profile       |
| GET    | /student/dashboard | STUDENT | Overview aggregation |

- The dashboard overview should be one aggregated call, not many parallel calls from the client. TODO: confirm current state.

Files: apps/api/src/student/

Known issues: TODO

## Learning

Modules: courses, certificates, projects, discussion, reviews

### courses

Purpose: The catalog and the learning content tree — course -> section -> video / quiz — plus progress and unlocking.

Owns models: Course, Section, Video, Quiz, QuizQuestion, progress models

Depends on: vdocipher (playback OTP), upload (thumbnails)

| Method | Route                        | Auth    | Purpose                           |
|--------|------------------------------|---------|-----------------------------------|
| GET    | /courses                     | Public  | Catalog with filters + pagination |
| GET    | /courses/:slug               | Public  | Course detail                     |
| GET    | /courses/:id/curriculum      | STUDENT | Sections + videos + lock state    |
| POST   | .../videos/:videoid/progress | STUDENT | Save watch progress               |
| GET    | /courses/:id/quizzes         | STUDENT | Course quizzes                    |

- Sequential unlocking: a video unlocks only when the previous one is complete. TODO: define exactly (percentage watched? explicit mark-complete?).
- Quizzes may gate progression — TODO: confirm.
- Course completion triggers certificate issuance. See 10 Core Flows / Learning & Certification.
- Unenrolled users must never receive video playback OTPs.

Files: apps/api/src/courses/

Known issues: TODO

## certificates

Purpose: Issue, list and publicly verify course completion certificates.

Owns models: Certificate

| Method | Route                      | Auth    | Purpose             |
|--------|----------------------------|---------|---------------------|
| GET    | /certificates/me           | STUDENT | My certificates     |
| GET    | /certificates/verify/:code | Public  | Public verification |
| GET    | /certificates/:id/download | STUDENT | PDF download        |

- Auto-issued on 100% completion — TODO: confirm the exact trigger point.
- Certificate codes must be unguessable (no sequential IDs in public URLs).
- Re-issuing must not create duplicates for the same user + course.

Files: apps/api/src/certificates/

## projects

Purpose: Live Projects assigned to and submitted by students.

Owns models: Project

- TODO — who assigns, who reviews, what states a project moves through.

Files: apps/api/src/projects/

## discussion

Purpose: Course discussion forum — posts, replies, likes.

Owns models: TODO: DiscussionPost, DiscussionReply, DiscussionLike

- Only enrolled students can post in a course's forum. TODO: confirm.
- Moderation: who can delete a post? TODO.
- TODO: notification on reply?

Files: apps/api/src/discussion/

## reviews

Purpose: Course ratings and written reviews.

Owns models: Review

- Only enrolled students may review; one review per user per course. TODO: confirm.
- TODO: are reviews moderated before publishing?
- Average rating shown on the catalog — TODO: computed live or stored?

Files: apps/api/src/reviews/

## Commerce

Modules: cart, checkout, coupon, refund, invoices, payment-settings, admin-payments. This is the highest-risk area of the codebase — document it first and most carefully.

### cart

Purpose: Pre-purchase basket and saved/wishlist items, for both guests and logged-in students.

Owns models: Cart, CartItem, Wishlist, WishlistItem — TODO: confirm names

- Guest cart lives in localStorage and merges into the DB cart on login. Merge must de-duplicate.
- A course the student is already enrolled in cannot be added.
- Price is not frozen in the cart — the authoritative price is resolved at checkout.

Files: apps/api/src/cart/

### checkout

Purpose: Create Razorpay orders, verify payments, handle webhooks, finalise enrollment.

Owns models: Order, OrderItem

Depends on: cart, coupon, payment-settings, invoices, mail

| Method | Route             | Auth      | Purpose                              |
|--------|-------------------|-----------|--------------------------------------|
| POST   | /checkout/order   | STUDENT   | Create Razorpay order from cart      |
| POST   | /checkout/verify  | STUDENT   | Verify client-side payment signature |
| POST   | /checkout/webhook | Signature | Razorpay server callback             |

- finalizeOrder() must be idempotent. Both /verify and the webhook call it; whichever arrives first wins and the second is a no-op.
- Never trust the amount sent by the client. Recompute server-side from course prices + coupon.
- Webhook signature is verified against the raw body. Raw body parsing must stay enabled in main.ts.
- Enrollment is created only inside finalizeOrder(), never from the client.

Files: apps/api/src/checkout/checkout.controller.ts, checkout.service.ts

### coupon

Purpose: Admin-created discount codes and their validation at checkout.

Owns models: Coupon

- Validation is a shared service consumed by checkout — never duplicated client-side.
- Rules to document precisely: expiry, usage limit (global and per user), minimum order value, course restrictions, currency applicability, stacking.

Files: apps/api/src/coupon/

## refund

Purpose: Refund requests and processing.

Owns models: Refund

- Refund window: TODO (days, and measured from what event).
- Approval flow: who can approve? Is a Razorpay refund API call made, or is it manual?
- On refund: is the enrollment revoked? Is an issued certificate revoked? Decide and document.

Files: apps/api/src/refund/

## invoices

Purpose: Invoice records and downloadable invoice documents.

- TODO — numbering scheme, tax fields (GST?), regeneration policy.

Files: apps/api/src/invoices/

## payment-settings

Purpose: Admin-controlled payment configuration.

- Domestic (INR) payments toggle — default state: TODO: confirm; intended default is OFF.
- Currency availability (INR / USD) per course.
- Changing settings must be audit-logged.

Files: apps/api/src/payment-settings/

## admin-payments

Purpose: Staff-side payment oversight — reconciliation, manual interventions, reporting.

- TODO — especially any endpoint that can create an enrollment without payment. Those must be audit-logged and ADMIN-only.

Files: apps/api/src/admin-payments/

## Content & CMS

Modules: blog, careers, legal-pages, contact, ai

## **blog**

Purpose: Articles shown at /articles, including AI-generated drafts.

Owns models: TODO: Article / BlogPost

- Draft -> review -> published states. AI-generated content must never auto-publish without human approval. TODO: confirm current behaviour.
- Slug uniqueness and SEO fields (title, description, OG image).

Files: apps/api/src/blog/

## **careers**

Purpose: Job postings at /careers and applications.

Owns models: JobPosting

- Only CONTENT\_MANAGER / ADMIN can create postings.
- Applications carry personal data and CVs — document retention and who can read them.

Files: apps/api/src/careers/

## **legal-pages**

Purpose: Admin-editable policy pages — Terms, Privacy, Refund, Cookie.

Owns models: LegalPage

- Fetched publicly by slug; slugs are fixed and must not change once linked from the footer.
- TODO: is version history kept? For legal pages it should be.
- Content is HTML/rich text — sanitise on render to avoid stored XSS.

Files: apps/api/src/legal-pages/

## **contact**

Purpose: Public contact form submissions.

- Public endpoint — must be rate limited and spam-protected. TODO: captcha?
- Notifies staff by email. TODO: which address.

Files: apps/api/src/contact/

## **ai**

Purpose: OpenAI-backed blog article generation.

- Only CONTENT\_MANAGER / ADMIN may trigger generation.
- Output is saved as a draft, never published directly.

- Cost control: token/request limits per day. TODO: implemented?
- Failures must degrade gracefully — an OpenAI outage must not break the blog.

Files: apps/api/src/ai/

Known issues: See also 13 AI Content Generation

## Staff & Operations

Modules: admin, sales, sales-targets, coordinator, trainer, support

### admin

Purpose: Platform administration — users, master data, global configuration.

- Every destructive admin action should write to the audit module.
- Role changes must be ADMIN-only and logged.

Files: apps/api/src/admin/

### sales

Purpose: Sales pipeline — leads, follow-ups, conversions.

- TODO — lead sources, assignment rules, what a SALES user may see about a student.
- Privacy note: define exactly which student fields a SALES user can read. Do not leave this implicit.

Files: apps/api/src/sales/

### sales-targets

Purpose: Targets per salesperson per period, and attainment reporting.

- TODO — period definition, who sets targets, how attainment is calculated (order created vs paid vs refund-adjusted).

Files: apps/api/src/sales-targets/

### coordinator

Purpose: TODO — one sentence on what a coordinator actually does in FutureStack.

Files: apps/api/src/coordinator/

### trainer

Purpose: Trainer-facing views — assigned courses, student progress, doubts.

- TODO — can a trainer edit course content, or only view?

Files: apps/api/src/trainer/

## support

Purpose: Support tickets raised by students and handled by SUPPORT staff.

Owns models: SupportTicket

- Ticket states: TODO (e.g. OPEN -> IN\_PROGRESS -> RESOLVED -> CLOSED).
- Assignment: automatic or manual? TODO.
- A student may only read their own tickets — enforce in the service, not just the guard.
- TODO: email notification on reply?

Files: apps/api/src/support/

## Platform

Modules: upload, vdocipher, mail, audit, common

### upload

Purpose: File uploads — course thumbnails, CVs, attachments.

- Allowed MIME types and max size: TODO.
- Storage target: TODO (S3 / Supabase Storage / local).
- Uploaded filenames must be randomised — never trust the client filename.
- Public vs signed-URL access: TODO.

Files: apps/api/src/upload/

### vdocipher

Purpose: Secure video playback — issues short-lived OTPs, receives VdoCipher webhooks.

- An OTP is issued only if the requesting user is enrolled in the course that owns the video — the single most important check in the module.
- OTP lifetime: TODO.
- Webhook signature verification: TODO — confirm. If missing, log as Critical in 15 Known Issues.

Files: apps/api/src/vdocipher/

### mail

Purpose: Transactional email.

- Email failures must not roll back the business transaction (an email failure should never block enrollment).
- TODO: queued or sent inline? Inline sending blocks the request.

Files: apps/api/src/mail/

### **audit**

Purpose: Records privileged actions for accountability.

- What is logged: TODO — list the exact actions.
- Audit entries are append-only. No update or delete endpoints.
- Readable by ADMIN only.

Files: apps/api/src/audit/

### **common**

Purpose: Shared building blocks used by every other module.

- @Auth(Role.X) — composed JwtAuthGuard + RolesGuard decorator.
- @CurrentUser() — injects the authenticated user.
- Guards: JwtAuthGuard, RolesGuard. Global exception filter: TODO — exists?
- Rule: anything imported by three or more modules belongs here, not copy-pasted.

Files: apps/api/src/common/

---

## **09 — Frontend Structure**

Owner: Ram · Last reviewed: 2026-09-19 · Status: Extracted from live route tree + proxy.ts + BFF proxy route

**Route tree — confirmed (find apps/web/app -name "page.tsx", 29 files)**

```

apps/web/app/
|-- (student)/
|   |-- (main)/
|   |   |-- about/page.tsx
|   |   |-- articles/page.tsx, articles/[slug]/page.tsx
|   |   |-- careers/page.tsx
|   |   |-- certificates/page.tsx
|   |   |-- courses/page.tsx, courses/[slug]/page.tsx
|   |   |-- faq/page.tsx
|   |   |-- live-projects/page.tsx, live-projects/[id]/page.tsx
|   |   |-- order-history/page.tsx
|   |   |-- research-and-development/page.tsx
|   |   `-- support/page.tsx (+ tickets/page.tsx, tickets/[id]/page.tsx)
|   |-- my-dashboard/page.tsx
|   |-- profile/page.tsx
|   `-- page.tsx                (home – embedded student login)
|-- auth/
|   |-- forgot-password/page.tsx
|   |-- oauth/callback/page.tsx
|   |-- reset-password/page.tsx
|   |-- set-new-password/page.tsx
|   `-- staff-login/page.tsx
|-- cart/page.tsx
`-- ops/
    |-- admin/page.tsx
    |-- content-manager/page.tsx
    |-- coordinator/page.tsx
    |-- sales/page.tsx
    |-- support/page.tsx
    `-- trainer/page.tsx

```

## Route protection — apps/web/proxy.ts

Despite the filename, this is the Next.js middleware convention file. It does exactly two things:

1. CSP with a per-request nonce on every page response. Notably permissive connect-src/frame-src/media-src for .razorpay.com, .vdocipher.com, and .s3-accelerate.amazonaws.com (the VdoCipher admin upload flow gets a signed URL from VdoCipher's API but the actual file upload PUT/POST goes straight to S3's accelerate endpoint from the browser).
2. A coarse gate on /ops/\* — if the fs\_token\_staff cookie is absent, redirect to /auth/staff-login?redirect=<original path>. This is UX only — "the backend still enforces real authorization on every API call."

Matcher excludes /api, \_next/static, \_next/image, favicon.ico, robots.txt, sitemap.xml, and prefetch requests (so a nonce is never baked into a reused RSC payload).

## The API layer — no single client, a BFF proxy instead

apps/web/app/api/[...proxy]/route.ts — every /api/\* call from the frontend hits this Next.js route, which forwards to the real NestJS backend (API\_URL, server-only env var, default http://localhost:3002). It:

- Strips the /api prefix and forwards method/body/query as-is.
- Forwards content-type and authorization headers if present.
- If no explicit authorization header, falls back to reading the fs\_token or fs\_token\_staff HttpOnly cookie and sets Authorization: Bearer <token> itself.
- Forwards x-forwarded-for/x-real-ip so the backend (behind Render's own LB) can log the true client IP.
- Forwards the raw cookie header too, so the backend can read refresh-token cookies directly.
- Applies different caching per request: public catalog GET requests (path contains /public/) get a 5-minute Next.js Data Cache; everything else is no-store.

This means the "central client" is really this one proxy route plus whatever fetch() calls individual components make against /api/... paths — there is no shared typed API client wrapping every call.

## Cart — confirmed dead feature

POST /cart/merge exists on the backend, is fully implemented, requires Role.STUDENT, and is designed (per its own docstring) to run automatically on login — but no code in apps/web calls it, and there is no local/guest cart storage to merge from. Practical consequence: an unauthenticated visitor cannot add anything to a cart today. Tracked as 15 Known Issues FS-010.

## Data fetching pattern

No single library confirmed dominant from this pass — components appear to call the BFF proxy directly via fetch(), with caching behavior split between the proxy route's own rules (above) and whatever each component does client-side.

# 10 — Core Flows

Owner: Ram · Last reviewed: TODO · Status: Completed

End-to-end journeys that cross multiple modules. Module pages explain parts; these describe the whole.

## Sub-flows documented

- Purchase & Enrollment
- Learning & Certification

- Quiz & Skill Test
- Refund
- Support Ticket
- Blog Generation & Publishing

## Flow page template (for adding new flows)

```
## Trigger
## Sequence diagram
## Step-by-step (with file paths)
## Data written
## Failure cases and recovery
## Open questions
```

# 11 — Commerce & Payments

Owner: Ram · Last reviewed: 2026-09-19 · Status: Completed

## Order creation — `createOrder()` (`checkout.service.ts:68-348`)

1. Currency legality is enforced server-side from `PaymentSettings`, regardless of what the client sends.
2. Cart items are re-validated live (still active, not already enrolled). Dropped items are removed from the cart and returned in a 409 so the frontend can show what changed.
3. Price is always recomputed server-side from the live Course/Project record — comment in the code states this explicitly: "Live prices from Course/Project — never trust the frontend."
4. Double-charge guard: a second CREATED order cannot be opened for a course that's already in another open order for the same user.
5. Coupon is re-validated from scratch at order-creation time — never trusted from whatever was cached on the cart.
6. GST is computed server-side from admin-configurable `PaymentSettings.gstPercent/gstPercentUsd` and snapshotted onto the Order row, so a later admin rate change never retroactively changes a historical invoice.
7. A minimum order amount (Rs.1 / \$0.50) is enforced so a 100%-discount coupon can't produce a Razorpay-rejected near-zero order.
8. Order + OrderItem rows are created inside a Prisma \$transaction, using a client-generated UUID as a placeholder `razorpayOrderId` before the real Razorpay order exists — then overwritten once Razorpay

responds.

9. If the Razorpay `orders.create()` call itself fails, the local Order row is deleted (not left dangling in a broken state).

## Order finalization — `finalizeOrder()` (`checkout.service.ts:387-535`)

This is the single most important function in the codebase — it is called from both `/checkout/verify` (client-initiated) and the `payment.captured` webhook (Razorpay-initiated), and must produce the same result no matter which one wins the race or whether both fire.

Idempotency mechanism: a single `updateMany({ where: { id: orderId, status: 'CREATED' }, data: { status: 'PAID', ... } })`. Only the first caller actually flips the row; Prisma reports `updated.count === 1` for the winner and 0 for the loser. The loser path is a safe no-op that just backfills `paymentMethod` if it was still null, then returns the enrollments the winner already created.

On the winning call, in order:

1. Create one Enrollment per course OrderItem on the order.
2. For each enrollment with a trainer, compute the split (`resolveTrainerSharePercent` — per-trainer override else global `PaymentSettings.trainerSharePercent`) and write one RevenueLedger row.
3. Flip any pending project OrderItems on the same order to active.
4. If a coupon was used: increment `Coupon.usedCount` and create a `CouponRedemption` row — this is the actual moment `perUserLimit` becomes durable; validation only checks existing redemptions, it doesn't reserve one.
5. Clear the user's cart.
6. Auto-attribution: if the order has no salesperson yet, look for an open (New/Interested) Lead matching this student (by `studentId` or email). If found and it has a salesperson, credit the order to that salesperson and auto-convert the lead.

All of the above happens inside one `$transaction` — either all of it commits or none of it does.

## Payment signature verification — `verifyPayment()`

Standard Razorpay pattern: HMAC-SHA256 of `${razorpayOrderId}${razorpayPaymentId}` using the account's key secret, compared against the client-supplied `razorpay_signature` with `crypto.timingSafeEqual` (constant-time, avoids timing side-channels). A mismatch is logged as a warning and rejected with a generic 400 — never leaks which part of the check failed.

## Razorpay webhook

POST `/webhooks/razorpay` — no auth guard (correct for a webhook); protection is entirely the signature check, verified against the raw request body before any JSON parsing happens. Handles

payment.captured (calls finalizeOrder()), payment.failed (CREATED -> FAILED), and payment.refunded (idempotent safety-net; the authoritative refund record normally comes from the admin-initiated /admin/refunds flow instead). The handler never throws to the caller — everything is caught and only logged, so Razorpay never receives a non-2xx and enters a retry storm.

## Is the client-sent amount ever trusted?

No, confirmed. Amount, currency legality, item prices, and GST are all recomputed server-side from Course/Project + PaymentSettings inside createOrder(). The client only ever sends back razorpay\_order\_id/razorpay\_payment\_id/razorpay\_signature for verification, and that verification is HMAC-based, not amount-based.

## PaymentSettings — current defaults

| Setting              | Default | Meaning                                                                            |
|----------------------|---------|------------------------------------------------------------------------------------|
| domesticEnabled      | true    | INR checkout is on by default                                                      |
| internationalEnabled | false   | USD checkout is off by default                                                     |
| trainerSharePercent  | 50      | Trainer keeps 50% of gross unless overridden per-trainer                           |
| gstPercent           | 18      | GST on INR orders                                                                  |
| gstPercentUsd        | 0       | Export-of-services zero-rating assumption                                          |
| usdRate              | 95      | INR per USD fallback conversion (used only when a course has no explicit priceUsd) |

updateSettings() explicitly refuses to leave both domesticEnabled and internationalEnabled false: "At least one payment method must remain enabled."

## Coupon validation — full rule order (coupon.service.ts:38-95)

1. Coupon exists (lookup by uppercased, trimmed code)
2. isActive
3. validFrom / validUntil window
4. Global maxUses vs usedCount
5. Per-user limit — via a live count of CouponRedemption rows, not a cached counter
6. Course/project restriction — applicableCourseIds empty array means "applies to everything"
7. minOrderAmount vs live subtotal
8. Currency restriction — only enforced for FLAT discount type (PERCENT coupons have no currency restriction at all).

Discount computation is done in integer minor units (Math.round(major \* 100)) specifically to avoid float

error at this step. PERCENT rounds down (`Math.floor`); FLAT is capped at the subtotal so a coupon can never produce a negative total by itself.

No coupon stacking exists — the data model only has room for one `couponId` per cart/order, so this is architectural, not a validated business rule.

## Refund — real behavior, not the aspirational state machine

`RefundStatus` enum has three values (PENDING, PROCESSED, REJECTED), but the code only exercises two transitions: (no row) -> PROCESSED (the normal admin-initiated create path writes the row directly as PROCESSED, no PENDING-on-create step), and PENDING -> REJECTED (via `rejectRefund()`, guarded). PENDING is never actually set anywhere in `refund.service.ts` — the guard exists, but nothing transitions a refund into that state. Both PROCESSED and REJECTED are terminal; there is no reversal path.

Duplicate-refund guard: a `findFirst` checks for an existing refund on the same (`orderId`, `courseId|projectId`) with status PENDING or PROCESSED before allowing a new one.

Razorpay call vs manual: gated by whether `order.razorpayOrderId` starts with "MANUAL-". Manual (offline sales) orders skip the Razorpay API entirely. Online orders call `payments.refund()` after first validating the requested amount against the live captured amount on Razorpay.

Side effects on refund, all inside one `$transaction`:

- Course refund -> the matching `Enrollment.status` flips to 'refunded' (soft-revoke), and its `RevenueLedger` row is deleted outright.
- Project refund -> the matching `OrderItem.status` flips to 'refunded'.
- The parent `Order.status` flips to 'REFUNDED' only once every course enrollment and project item on that order has been individually refunded — a partial refund leaves the order PAID.
- Certificate revocation: confirmed NOT implemented. A refund never touches the `Certificate` table.
- Coupon usage restoration: confirmed NOT implemented. `usedCount` and `CouponRedemption` rows are never reversed by a refund.

See 15 Known Issues FS-005 and FS-006 for these last two.

## Money precision

Every monetary field in the schema is `Float`. The coupon-discount calculation defends against this locally by working in integer minor units — but GST computation, revenue-ledger splits, and invoice totals are not confirmed to use the same defensive pattern. See 05 Database Schema and 15 Known Issues FS-001.

# 12 — Admin CMS & Master Data

Owner: Ram · Last reviewed: TODO · Status: Completed

What staff can change from the ops panel without a code deploy.

## Editable content

| Area                                                | Model                | Console              | Who             |
|-----------------------------------------------------|----------------------|----------------------|-----------------|
| Terms of Service / Privacy / Refund / Cookie policy | LegalPage            | /ops/admin -> Pages  | ADMIN           |
| Articles / blog                                     | TODO                 | /ops/content-manager | CONTENT_MANAGER |
| Career job postings                                 | JobPosting           | /ops/content-manager | CONTENT_MANAGER |
| Payment settings                                    | TODO                 | /ops/admin           | ADMIN           |
| Master data                                         | TODO                 | /ops/admin           | ADMIN           |
| Courses & curriculum                                | Course/Section/Video | /ops/admin           | ADMIN / TRAINER |
| FAQ, Homepage content                               | TODO                 |                      |                 |

## Legal pages

- Slugs are referenced by the public footer and must never change: terms, privacy, refund, cookie — TODO: confirm actual slugs.
- Rich text is stored as HTML — sanitise on render.
- TODO: keep version history. For legal content, being able to prove what the policy said on a given date matters.

## Master data

TODO: list what "master data" actually contains — categories, tags, skill levels, cities, lead sources, etc. For each: model, who edits it, and what breaks if a value is deleted while in use.

## CMS rules

1. Deleting master data that is referenced by existing records must be blocked, not cascaded.
2. Every CMS change writes an audit entry (who, what, when, before/after where feasible).
3. Public pages that read CMS content should be cached.
4. Content managers cannot change anything that affects money or access control.

# 13 — AI Content Generation

Owner: Ram · Last reviewed: TODO · Status: Completed

Blog articles are generated with OpenAI through the ai module. The end-to-end flow lives in 10 Core Flows / Blog Generation & Publishing — this page covers configuration and operating rules.

## Configuration

| Item                   | Value                       |
|------------------------|-----------------------------|
| Provider               | OpenAI                      |
| Model                  | TODO                        |
| API key env var        | OPENAI_API_KEY              |
| Max tokens per request | TODO                        |
| Daily request cap      | TODO — implement if missing |
| Timeout                | TODO                        |
| Retry policy           | TODO                        |

## Prompt design

TODO: paste the actual system/user prompt template used, and keep it in this page. Prompts are product logic — they deserve version control and review like code.

```
System: ...  
User: ...
```

Expected output format: TODO (JSON with title, slug, excerpt, body, SEO fields?)

## Operating rules

1. Generated content is saved as a draft. A human publishes it.
2. Generation is triggered only by CONTENT\_MANAGER / ADMIN.
3. Every generated article is flagged in the DB so AI content can be audited later.
4. Facts in generated articles are not trusted. See reviewer checklist below.
5. If OpenAI is unavailable, the blog continues serving existing articles — generation failure is never user-facing.

## Reviewer checklist before publishing

- & Facts and any numbers verified
- & No hallucinated citations, people, or product claims
- & Brand voice and formatting consistent
- & Slug, meta title, meta description present and unique
- & Internal links added where relevant
- & No duplicate of an existing article

## Cost control

| Control                  | Status |
|--------------------------|--------|
| Per-day request cap      | TODO   |
| Per-request token cap    | TODO   |
| Usage logging            | TODO   |
| Alert on spend threshold | TODO   |

## Open questions

- TODO: scheduled auto-generation, or manual only?
- TODO: is there a separate pipeline (e.g. n8n) planned or running alongside the ai module? If so, document which one is authoritative so the two do not both publish.

# 14 — Deployment

Owner: Ram · Last reviewed: 2026-09-19 · Status: Completed

## Hosting topology

| Service                     | Platform                                               | Evidence                                                                              |
|-----------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------|
| apps/api (NestJS backend)   | Render — single web service futurestack-api, free plan | render.yaml at repo root                                                              |
| apps/web (Next.js frontend) | Not defined in this repo — almost certainly Vercel     | Code comments reference "Vercel's LB"; no vercel.json (zero-config Next.js detection) |
| Database                    | Supabase Postgres                                      | DATABASE_URL + DIRECT_URL (pooled vs direct connection)                               |

## render.yaml — key contents

```
services:
  - type: web
    name: futurestack-api
    runtime: node
    plan: free
    branch: main
    buildCommand: npx pnpm install && npx pnpm --filter api exec prisma generate &&
npx pnpm --filter api build
    startCommand: npx pnpm --filter api start:prod
    envVars:
      - key: PORT
        value: 3002
      - key: DATABASE_URL
        sync: false
      - key: JWT_SECRET
        generateValue: true
      - key: JWT_EXPIRES_IN
        value: 7d
      # ...GOOGLE_*, AGENTMAIL_*, CONTACT_EMAIL, VDOCIPHER_API_KEY
```

Free plan — worth knowing this implies possible cold starts and no horizontal scaling by default.

## Build & start commands

Root package.json (entire file — it's small):

```
{
  "name": "future-stack",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev:web": "pnpm --filter web dev",
    "dev:api": "pnpm --filter api dev"
  }
}
```

apps/api/package.json scripts: build "nest build", start "nest start", start:dev "nest start --watch", start:prod "node dist/src/main", db:seed "ts-node prisma/seed.ts", lint, test, test:cov, test:e2e.

apps/web/package.json scripts: dev "next dev" (with memory/DNS flags), build "next build", start "next start", lint "eslint".

Render's startCommand (pnpm --filter api start:prod -> node dist/src/main) matches the build output path from nest build.

## Migrations in production

```
npx prisma migrate deploy # never migrate dev in production
```

Take a Supabase backup before any destructive migration. `DIRECT_URL` (bypassing the pooler) is what Prisma migrations should run against — confirm this is how `render.yaml`'s build step is actually invoking Prisma if migrations are run as part of deploy (the current `buildCommand` only runs `prisma generate`, not `migrate deploy` — confirm how/when migrations are actually applied in this pipeline).

## Monitoring

`GET /healthz` exists and is explicitly built for this: returns `{status, dbOk, dbLatencyMs, uptimeSec, timestamp}`, `unauthenticated`, `@SkipThrottle()`. Used both by Render's own health checks and the admin System Health view. Point any external uptime monitor at this endpoint.

# 15 — Glossary

Shared vocabulary. If two people use the same word for different things, decide here.

| Term         | Means in FutureStack                                                                                |
|--------------|-----------------------------------------------------------------------------------------------------|
| Course       | The sellable learning product. Contains sections.                                                   |
| Section      | An ordered group of videos and quizzes inside a course.                                             |
| Video        | One lesson, hosted on VdoCipher.                                                                    |
| Quiz         | A set of questions. Course-linked or standalone (then shown as a Skill Test).                       |
| Skill Test   | A standalone quiz with no course link, taken from the dashboard.                                    |
| Attempt      | One submission of a quiz by a student, with a score.                                                |
| Enrollment   | A user's paid access to a course. Created only after payment is finalised.                          |
| Order        | A checkout attempt and its payment state.                                                           |
| Order Item   | One course within an order, with the price at purchase.                                             |
| Finalisation | The idempotent step that flips an order to PAID and creates enrollments.                            |
| Certificate  | Proof of course completion, publicly verifiable by code.                                            |
| Live Project | Real project work a student does alongside a course.                                                |
| Ops panel    | The staff area at <code>/ops</code> , split into role-based consoles.                               |
| Console      | One role's section of the ops panel (admin, sales, trainer, support, coordinator, content manager). |
| Master data  | Admin-editable reference values used in dropdowns and configuration.                                |

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| Legal page        | An admin-editable policy page (Terms, Privacy, Refund, Cookie). |
| Article           | A blog post at /articles, possibly AI-generated.                |
| Support ticket    | A student support request and its reply thread.                 |
| Staff             | Any user whose role is not STUDENT.                             |
| Domestic payments | INR payments, controlled by an admin toggle.                    |